Data Structures (CSN12101)
ASSIGNMENT-9

Name : Aayansh Niranjan Date :01/04/2026
Reg_No.: 20253004

Q1. Implement a binary tree, supporting the following operations:

a) Insert(): Insert a node into the tree.

b) Display(): Display the tree using in-order traversal.

A1. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node{
    int data;
    struct Node *left,*right;
};

struct Node* createNode(int data){
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=data;
    newNode->left=newNode->right=NULL;
    return newNode;
}

struct Node* insert(struct Node* root,int data){
    if (root==NULL)
        return createNode(data);
    if (data<root->data)
        root->left=insert(root->left,data);
    else
        root->right=insert(root->right,data);
    return root;
}
void inorder(struct Node* root){
    if(root!=NULL){
        inorder(root->left);
        printf("%d ",root->data);
        inorder(root->right);
    }
}
int main(){
    struct Node* root=NULL;
    root=insert(root,50);
    insert(root,30);
    insert(root,70);
    insert(root,20);
    insert(root,40);
    printf("In-order traversal: ");
    inorder(root);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_9$ ./p1
In-order traversal: 20 30 40 50 70
```

Q2. Implement a binary search tree (BST) with the following operations:

a) Insert(): Insert a node in the BST.

b) Search(): Search for a given element in the BST and return whether it exists.

c) Delete(): Delete a node from the BST.

A2. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node{
    int data;
    struct Node *left,*right;
};

struct Node* createNode(int data){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->data=data;
    n->left=n->right=NULL;
    return n;
}

struct Node* insert(struct Node* root,int data){
    if(!root) return createNode(data);
    if(data<root->data)
        root->left=insert(root->left,data);
    else if (data>root->data)
        root->right=insert(root->right,data);
    return root;
}

int search(struct Node* root,int key){
    if(!root) return 0;
    if(root->data==key) return 1;
    if(key<root->data)
        return search(root->left,key);
    else
        return search(root->right,key);
}

struct Node* findMin(struct Node* root){
    while(root->left)
        root=root->left;
    return root;
}

void inorder(struct Node* root){
    if(root){
        inorder(root->left);
        printf("%d ",root->data);
        inorder(root->right);
    }
}
```

```
struct Node* deleteNode(struct Node* root, int key) {
    if(!root) return NULL;
    if(key<root->data)
        root->left=deleteNode(root->left,key);
    else if(key>root->data)
        root->right=deleteNode(root->right,key);
    else{
        if(!root->left){
            struct Node* temp=root->right;
            free(root);
            return temp;
        }
        else if(!root->right){
            struct Node* temp=root->left;
            free(root);
            return temp;
        }
        struct Node* temp=findMin(root->right);
        root->data=temp->data;
        root->right=deleteNode(root->right,temp->data);
    }
    return root;
}

int main(){
    struct Node* root=NULL;
    root=insert(root,50);
    insert(root,30);
    insert(root,70);
    insert(root,20);
    insert(root,40);
    printf("Tree: ");
    inorder(root);
    printf("\nSearch 40: %d",search(root, 40));
    root=deleteNode(root,30);
    printf("\nAfter deletion: ");
    inorder(root);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_9$ ./p2
Tree: 20 30 40 50 70
Search 40: 1
After deletion: 20 40 50 70
```

Q3. Implement the three types of depth-first traversal methods in a binary tree:

a) In-order Traversal: Left -> Root -> Right

b) Pre-order Traversal: Root -> Left -> Right

c) Post-order Traversal: Left -> Right -> Root

A3. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

struct Node* newNode(int data){
   struct Node* n=(struct Node*)malloc(sizeof(struct Node));
   n->data=data;
   n->left=n->right=NULL;
   return n;
}

void inorder(struct Node* root){
   if(root){
      inorder(root->left);
      printf("%d ",root->data);
      inorder(root->right);
   }
}

void preorder(struct Node* root){
   if(root){
      printf("%d ",root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

void postorder(struct Node* root){
   if(root){
      postorder(root->left);
      postorder(root->right);
      printf("%d ",root->data);
   }
}

int main(){
   struct Node* root=newNode(1);
   root->left=newNode(2);
   root->right=newNode(3);
   printf("In-order: ");
   inorder(root);
   printf("\nPre-order: ");
   preorder(root);
   printf("\nPost-order: ");
   postorder(root);
   printf("\n");
   return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_9$ ./p3
In-order: 2 1 3
Pre-order: 1 2 3
Post-order: 2 3 1
```

Q4. Write a C program to calculate the height of a binary tree. Ensure the program handles empty trees situation.

A4. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node{
    int data;
    struct Node *left,*right;
};

struct Node* newNode(int data){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->data=data;
    n->left=n->right=NULL;
    return n;
}

int height(struct Node* root){
    if(root==NULL)
        return 0;
    int left=height(root->left);
    int right=height(root->right);
    return(left>right ? left:right)+1;
}

int main(){
    struct Node* root=newNode(1);
    root->left=newNode(2);
    root->right=newNode(3);
    root->left->left=newNode(4);
    printf("Height of tree: %d",height(root));
    printf("\n");
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_9$ ./p4
Height of tree: 3
```

Q5. Write a C program to perform level-order traversal (breadth-first traversal) of a binary tree. In level-order traversal, nodes are visited level by level from left to right.

A5. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left,*right;
};

struct Node* newNode(int data){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->data=data;
    n->left=n->right=NULL;
    return n;
}

void levelOrder(struct Node* root){
    if (!root) return;
    struct Node* queue[100];
    int front=0,rear=0;
    queue[rear++]=root;
    while(front<rear){
        struct Node* temp=queue[front++];
        printf("%d ",temp->data);
        if(temp->left)
            queue[rear++]=temp->left;
        if(temp->right)
            queue[rear++]=temp->right;
    }
}

int main(){
    struct Node* root=newNode(1);
    root->left=newNode(2);
    root->right=newNode(3);
    root->left->left=newNode(4);
    printf("Level-order traversal: ");
    levelOrder(root);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_9$ ./p5
Level-order traversal: 1 2 3 4
```